System Design Document

TaskAnalyzer · Especificação SDD v1.0

Arthur Amaral dos Santos · ADS · CEUB · Bootcamp III

Especificação SDD — TaskAnalyzer (v1.0)

> Contrato executável do módulo TaskAnalyzer. Transposto da Fase 1 (Google Docs). O código só é aceito se passar em todos os cenários de aceite da Seção 2 e respeitar o CONTEXT_RULES.md. Qualquer comportamento não descrito aqui é **não especificado** e não deve ser implementado.

1. Visão Geral e Contrato de Negócio (SDD)

1.1 Identificação

Nome completo	**Arthur Amaral Dos Santos**
Curso	Analise e Desenvolvimento de Sistemas
Turma	Turma A - 0726
E-mail institucional	arthur.amaral@sempreceub.com
Disciplina	Bootcamp III (EAD — Disciplina Virtual)
Data da especificação	11/09/2026
Versão da especificação	1.0

1.2 Propósito do Módulo TaskAnalyzer

O **TaskAnalyzer** é um módulo avançado de métricas de produtividade. Ele recebe uma lista de tarefas já executadas ou em andamento e devolve um relatório com indicadores que apoiam a tomada de decisão de líderes e equipes: quanto tempo as tarefas levam para ser concluídas, com que frequência os prazos são estourados e como esse comportamento varia por prioridade.

Escopo (o que o módulo faz)

- Validar rigorosamente cada tarefa recebida (tipos, valores permitidos e coerência entre datas).
- Calcular o **tempo médio de conclusão**, em minutos, das tarefas concluídas — geral e por prioridade.
- Calcular a **taxa de atraso**, em percentual, das tarefas concluídas após o prazo — geral e por prioridade.
- Informar a **quantidade de tarefas** analisadas e concluídas, geral e por prioridade (baixa, média, alta).
- Sinalizar entradas inválidas por meio de **exceções específicas** com mensagens claras.

Fora do escopo (o que o módulo NÃO faz)

- Não persiste dados (sem arquivos, banco de dados ou cache).
- Não possui interface gráfica, API HTTP ou linha de comando.
- Não converte fusos horários: todas as datas já chegam em UTC.
- Não corrige, descarta ou "adivinha" dados inválidos — a análise é interrompida com exceção.

1.3 Contrato Executável de Interface

1.3.1 Assinatura pública

O módulo expõe **uma única função pública**. Nome, parâmetros e tipo de retorno não podem ser alterados sem nova versão desta especificação.

```python
# src/task_analyzer.py
def analisar_tarefas(tarefas: list[Tarefa]) -> RelatorioMetricas:
    """Analisa a lista de tarefas e retorna as métricas de produtividade."""
```

Tipos públicos definidos no mesmo módulo (apenas biblioteca padrão):

```python
@dataclass(frozen=True)
class Tarefa:
    id_tarefa: int
    titulo: str
    prioridade: str                 # "baixa" | "media" | "alta"
    status: str                     # "concluida" | "pendente" | "cancelada"
    data_criacao: datetime          # UTC, com fuso
    prazo: datetime                 # UTC, com fuso
    data_inicio: datetime | None = None
    data_conclusao: datetime | None = None


class IndicadorPrioridade(TypedDict):
    quantidade_concluidas: int
    tempo_medio_conclusao_min: float | None
    taxa_atraso_percentual: float | None


class RelatorioMetricas(TypedDict):
    total_tarefas: int
    total_concluidas: int
    tempo_medio_conclusao_min: float
    taxa_atraso_percentual: float
    indicadores_por_prioridade: dict[str, IndicadorPrioridade]
```

1.3.2 Entradas (campos de Tarefa)

Campo	Tipo	Obrigatório	Descrição / Restrições
id_tarefa	int	Sim	Identificador único na lista; inteiro > 0 (bool não é aceito).
titulo	str	Sim	Texto não vazio (não pode conter apenas espaços).
prioridade	str (enum)	Sim	Exatamente um de: baixa, media, alta (minúsculas, sem acento).
status	str (enum)	Sim	Exatamente um de: concluida, pendente, cancelada.
data_criacao	datetime	Sim	Data/hora de criação, com fuso (UTC).
prazo	datetime	Sim	Data/hora limite; deve ser ≥ data_criacao.
data_inicio	datetime | None	Condicional	Obrigatório se status = concluida; se informado, deve ser ≥ data_criacao.
data_conclusao	datetime | None	Condicional	Obrigatório se status = concluida e deve ser ≥ data_inicio. Deve ser None nos demais status.

Parâmetro tarefas: list[Tarefa] com pelo menos 1 elemento. A lista **não é modificada** pela função.

1.3.3 Saídas (RelatorioMetricas)

Campo	Tipo	Regra de cálculo
total_tarefas	int	Quantidade total de tarefas recebidas (todos os status).
total_concluidas	int	Quantidade de tarefas com status concluida.
tempo_medio_conclusao_min	float	Média de (data_conclusao − data_inicio) em minutos, apenas concluídas. 2 casas decimais.
taxa_atraso_percentual	float	(concluídas com data_conclusao > prazo) ÷ total_concluidas × 100. 2 casas decimais.
indicadores_por_prioridade	dict	Sempre com as 3 chaves baixa, media, alta, cada uma com quantidade_concluidas, tempo_medio_conclusao_min e taxa_atraso_percentual calculados só com as concluídas daquela prioridade.

Exemplo de retorno (resultado esperado do Cenário CA01):

```
{
  "total_tarefas": 6,
  "total_concluidas": 4,
  "tempo_medio_conclusao_min": 97.5,
  "taxa_atraso_percentual": 50.0,
  "indicadores_por_prioridade": {
    "baixa": {"quantidade_concluidas": 1,
              "tempo_medio_conclusao_min": 180.0,
              "taxa_atraso_percentual": 100.0},
    "media": {"quantidade_concluidas": 1,
              "tempo_medio_conclusao_min": 30.0,
              "taxa_atraso_percentual": 0.0},
    "alta":  {"quantidade_concluidas": 2,
              "tempo_medio_conclusao_min": 90.0,
              "taxa_atraso_percentual": 50.0}
  }
}
```

1.3.4 Exceções do contrato

Todas as exceções herdam de uma base comum, permitindo que o chamador trate qualquer erro do módulo com um único except TaskAnalyzerError.

Exceção	Herda de	Quando é lançada
TaskAnalyzerError	Exception	Base de todas as exceções do módulo (não é lançada diretamente).
EntradaInvalidaError	TaskAnalyzerError	Lista vazia ou que não é list; item que não é Tarefa; id_tarefa ≤ 0 ou duplicado; titulo vazio; prioridade ou status fora dos valores permitidos.
DataInvalidaError	TaskAnalyzerError	Datetime sem fuso horário; prazo < data_criacao; data_inicio < data_criacao; tarefa concluída sem data_inicio/data_conclusao; data_conclusao < data_inicio; tarefa não concluída com data_conclusao.
SemTarefasConcluidasError	TaskAnalyzerError	Nenhuma tarefa com status concluida na lista (evita divisão por zero).

Toda mensagem de exceção deve estar em português e citar o id_tarefa e o campo com problema quando aplicável. Ex.: Tarefa 7: data_conclusao anterior a data_inicio.

1.3.5 Regras de negócio e restrições

ID	Regra
RN01	Somente tarefas com status concluida entram no cálculo de tempo médio e taxa de atraso. Pendentes e canceladas contam apenas em total_tarefas.
RN02	Tempo de conclusão = data_conclusao − data_inicio, expresso em minutos (float).
RN03	Uma tarefa está atrasada quando data_conclusao > prazo. Concluir **exatamente** no prazo **não** é atraso.
RN04	Valores de tempo e percentual são arredondados para 2 casas decimais com round(valor, 2).
RN05	Todas as datas devem ser timezone-aware e estar em UTC. Datas sem fuso são rejeitadas (DataInvalidaError).
RN06	Validação fail-fast: a primeira tarefa inválida interrompe a análise inteira. Não existe resultado parcial e nenhuma tarefa é ignorada silenciosamente.
RN07	Coerência temporal obrigatória: data_criacao ≤ data_inicio ≤ data_conclusao e data_criacao ≤ prazo.
RN08	Se não houver nenhuma tarefa concluída, lança SemTarefasConcluidasError (proteção contra divisão por zero).
RN09	Prioridade sem tarefas concluídas **não é erro**: quantidade_concluidas = 0 e os campos de média e taxa retornam None.
RN10	id_tarefa deve ser único na lista recebida.
RN11	A função é pura: não altera a lista de entrada, não acessa rede, arquivos ou banco de dados.
RN12	Eventos relevantes (início da análise, total processado, erros de validação) são registrados com o módulo logging, nunca com print.

2. Especificação de Cenários de Aceite e Test Harness

Os cenários abaixo seguem o formato **Dado – Quando – Então** e são a referência objetiva de "código correto". Eles serão convertidos, sem alteração de valores, em testes automatizados com **pytest** no arquivo tests/test_harness.py.

2.1 Massa de dados padrão (MD01)

Todas as datas em **01/09/2026, UTC**. data_criacao = 07:00 para todas as tarefas.

ID	Título	Prioridade	Status	Início	Conclusão	Prazo	Tempo / Atraso
1	Configurar ambiente	alta	concluida	08:00	09:00	10:00	60 min / não
2	Corrigir bug de login	alta	concluida	08:00	10:00	09:00	120 min / sim
3	Revisar PR	media	concluida	09:00	09:30	12:00	30 min / não
4	Atualizar documentação	baixa	concluida	10:00	13:00	12:00	180 min / sim
5	Criar dashboard	media	pendente	11:00	—	18:00	ignorada
6	Migrar servidor	baixa	cancelada	—	—	18:00	ignorada

2.2 Cenários de aceite

Cenário CA01 — Sucesso: métricas corretas

Dado	**a massa de dados MD01 (6 tarefas válidas com diferentes prioridades, prazos e status)**
Quando	analisar_tarefas(MD01) for executado
Então	deve retornar exatamente: • total_tarefas = 6 e total_concluidas = 4 • tempo_medio_conclusao_min = 97.5 → (60 + 120 + 30 + 180) ÷ 4 • taxa_atraso_percentual = 50.0 → 2 atrasadas ÷ 4 concluídas • alta: 2 concluídas, 90.0 min, 50.0% | media: 1, 30.0 min, 0.0% | baixa: 1, 180.0 min, 100.0%
Teste	test_ca01_metricas_corretas_com_massa_padrao

Cenário CA02 — Exceção: datas inválidas

Dado	**uma tarefa id 7, status concluida, criada às 07:00, prazo 12:00, data_inicio = 10:00 e data_conclusao = 09:00 (conclusão antes do início)**
Quando	o analisador for executado com essa tarefa na lista
Então	deve lançar DataInvalidaError com mensagem contendo Tarefa 7 e data_conclusao, sem retornar nenhuma métrica
Teste	test_ca02_conclusao_antes_do_inicio_lanca_data_invalida

Cenário CA03 — Exceção: divisão por zero

Dado	**uma lista válida contendo apenas as tarefas 5 (pendente) e 6 (cancelada) da MD01**
Quando	o analisador for executado
Então	deve lançar SemTarefasConcluidasError com mensagem clara, e **nunca** ZeroDivisionError
Teste	test_ca03_sem_tarefas_concluidas_lanca_excecao_especifica

Cenário CA04 — Borda: arredondamento, prazo exato e prioridade vazia

Dado	**3 tarefas alta concluídas, criadas às 07:00 e iniciadas às 08:00: • id 1: conclusão 08:10, prazo 08:10 (no limite exato) • id 2: conclusão 08:10, prazo 09:00 • id 3: conclusão 08:11, prazo 08:10 (1 minuto atrasada)**
Quando	o analisador for executado
Então	• tempo_medio_conclusao_min = 10.33 → (10 + 10 + 11) ÷ 3 = 10,333... • taxa_atraso_percentual = 33.33 → só a tarefa 3 está atrasada (RN03) • baixa e media: quantidade_concluidas = 0 e médias/taxas = None (RN09)
Teste	test_ca04_arredondamento_prazo_exato_e_prioridade_vazia

Cenário CA05 — Exceção: entradas inválidas (parametrizado)

Dado	**cada uma das entradas: (a) lista vazia; (b) prioridade urgente; (c) status feito; (d) dois itens com id_tarefa = 1; (e) id_tarefa = 0; (f) titulo = " "**
Quando	o analisador for executado
Então	deve lançar EntradaInvalidaError em todos os casos
Teste	test_ca05_entradas_invalidas_lancam_entrada_invalida (com @pytest.mark.parametrize)

Cenário CA06 — Exceção: datas sem fuso ou incoerentes (parametrizado)

Dado	**cada uma das entradas: (a) data_criacao sem fuso; (b) prazo antes da criação; (c) concluída sem data_conclusao; (d) pendente com data_conclusao preenchida**
Quando	o analisador for executado
Então	deve lançar DataInvalidaError em todos os casos
Teste	test_ca06_datas_incoerentes_lancam_data_invalida (com @pytest.mark.parametrize)

Cenário CA07 — Função pura: entrada não é alterada

Dado	**uma cópia da massa MD01 guardada antes da execução**
Quando	o analisador for executado com MD01
Então	a lista MD01 deve permanecer idêntica à cópia (mesmos itens, mesma ordem)
Teste	test_ca07_lista_de_entrada_nao_e_modificada

2.3 Planejamento do Test Harness

O Test Harness é a "rede de segurança" que valida objetivamente o código gerado por IA. O fluxo é:

Etapa	O que acontece	Artefato
1. Cenários escritos	Cenários CA01–CA07 desta especificação definem entradas e resultados esperados.	specs/task_analyzer_spec.md
2. Testes automatizados	Cada cenário vira uma função test_caXX_... em pytest, **escrita antes do código** (TDD).	tests/test_harness.py
3. Geração do código	A IA recebe spec + CONTEXT_RULES + testes e gera a implementação.	src/task_analyzer.py
4. Validação	python -m pytest -v precisa passar 100%. Falhou? Corrige-se o código, **nunca** o teste.	Saída do pytest
5. Homologação humana	Revisão crítica conforme a Seção 4.2 antes do commit na main.	Pull Request aprovado

Técnicas pytest que serão usadas

- **Fixtures** (@pytest.fixture) para montar a massa MD01 e as datas UTC, evitando repetição.
- **pytest.raises(..., match=...)** para validar o tipo da exceção **e** o conteúdo da mensagem.
- **@pytest.mark.parametrize** para cobrir vários casos inválidos com um único teste (CA05 e CA06).
- Comparação exata dos valores do dicionário de retorno (não apenas "se existe").
- Comando de execução: python -m pytest -v tests/ — a Fase 3 executará o mesmo comando no GitHub Actions.

Esqueleto previsto para tests/test_harness.py:

```python
from datetime import datetime, timezone

import pytest

from src.task_analyzer import (Tarefa, analisar_tarefas, DataInvalidaError,
                               SemTarefasConcluidasError)


def utc(hora: int, minuto: int = 0) -> datetime:
    return datetime(2026, 9, 1, hora, minuto, tzinfo=timezone.utc)


@pytest.fixture
def massa_padrao() -> list[Tarefa]:
    ...  # tarefas 1 a 6 da MD01


def test_ca01_metricas_corretas_com_massa_padrao(massa_padrao):
    resultado = analisar_tarefas(massa_padrao)
    assert resultado["tempo_medio_conclusao_min"] == 97.5
    assert resultado["taxa_atraso_percentual"] == 50.0


def test_ca03_sem_tarefas_concluidas_lanca_excecao_especifica(massa_padrao):
    with pytest.raises(SemTarefasConcluidasError):
        analisar_tarefas(massa_padrao[4:])
```